

19 Querying JPA with Querydsl

This chapter covers

- Introducing Querydsl
- Creating a Querydsl application
- Querying a database with Querydsl

Querying a database is essential to retrieve information that meets certain criteria. This chapter focuses on Querydsl, one of the alternatives for querying a database from Java programs. The “dsl” part of the Querydsl name refers to domain specific languages (DSLs), which are languages dedicated to particular application domains. For example, querying a database is such a domain.

In this chapter we'll examine the most important capabilities of Querydsl, and we'll apply them in a Java persistence project. For comprehensive documentation of Querydsl, see its website: <http://querydsl.com/>.

19.1 Introducing Querydsl

There are various alternatives for querying a database from within Java programs. You can use SQL, as has been possible since the early days of JDBC. The drawbacks of this approach are the lack of portability (the queries are dependent on the database and the particular SQL dialect) and the lack of type safety and static query verification.

JPQL (Jakarta Persistence Query Language) was a step forward, being an object-oriented query language that is independent of the database. This means there's no lack of portability, but there's still a lack of type safety and static query verification.

Spring Data allows us to create methods with the query builder mechanism and to annotate methods with JPQL and SQL queries (though these still have their previously mentioned drawbacks). The query builder mechanism also has the disadvantage of requiring the methods to be defined in advance, and their names are not statically checked at compile time.

The Criteria API allows you to build type safe and portable queries using a Java API. While it solved the drawbacks of the previously presented al-

ternatives, it ended up being extremely verbose and created code that was difficult to read.

Querydsl keeps the important ideas of type safety and portability. Furthermore, it reduces the verbosity of Criteria API, and the code it creates is much easier to read and understand than that built with Criteria API.

19.2 Creating a Querydsl application

We'll start by creating a Querydsl application whose dependencies are managed by Maven. We'll examine the steps involved, the dependencies that need to be added to the project, the entities that will be managed, and how to write queries with the help of Querydsl.

NOTE To execute the examples from the source code, you'll first need to run the Ch19.sql script.

19.2.1 Configuring the Querydsl application

We'll add two dependencies in the Maven pom.xml file: `querydsl-jpa` and `querydsl-apt`. The `querydsl-jpa` dependency is needed to use the Querydsl API inside a JPA application. The `querydsl-apt` dependency is needed to process annotations from Java files before code compilation.

The APT in `querydsl-apt` stands for Annotation Processing Tool, and using it, the entities that are managed by the application will be replicated in the so-called Q-types (Q standing for “query”). This means that each `Entity` entity will have a corresponding `QEntity` that will be generated at build time, which Querydsl will use to query the database. Also, each field of the entity will be mirrored in the `QEntity` using the specific Querydsl classes. For example, `String` fields will be mirrored to `StringPath` fields, `Long` fields to `NumberPath<Long>` fields, `Integer` fields to `NumberPath<Integer>` fields, and so on.

The resulting Maven pom.xml file will have the dependencies shown in the following listing.

Listing 19.1 The pom.xml Maven file

```
Path: Ch19/querydsl/pom.xml

\1
    <groupId>com.querydsl</groupId>
    <artifactId>querydsl-jpa</artifactId>
    <version>5.0.0</version>
</dependency>
<dependency>
```

```

        <groupId>com.querydsl</groupId>
        <artifactId>querydsl-apt</artifactId>
        <version>5.0.0</version>
        <scope>provided</scope>
    </dependency>

```

The scope of the `querydsl-apt` dependency is specified as `provided`. This means that the dependency is needed only at build time when Maven generates the previously introduced Q-types. Then it is no longer needed, so it won't be included in the application artifacts.

To work with Querydsl we also need to include the Maven APT plugin in the Maven `pom.xml` file. This plugin will take care of generating the Q-types during the build process. As we are using JPA annotations in our project, the class that effectively does this is `com.querydsl.apt.jpa.JPAAAnnotationProcessor`. If we were using the Hibernate API and annotations, we would have to use `com.querydsl.apt.hibernate.HibernateAnnotationProcessor` instead.

We'll also have to indicate the output directory where the generated Q-types will reside: inside the `target` Maven folder. The `pom.xml` file with all this added is shown in the following listing.

Listing 19.2 The `pom.xml` Maven file with the APT plugin

Path: Ch19/querydsl/pom.xml

```

<plugin>
  <groupId>com.mysema.maven</groupId>
  <artifactId>apt-maven-plugin</artifactId>
  <version>1.1.3</version>
  <executions>
    <execution>
      <goals>
        <goal>process</goal>
      </goals>
      <configuration>
        <outputDirectory>target/generated-sources/java</outputDirectory>
        <processor>com.querydsl.apt.jpa.JPAAAnnotationProcessor</processor>
      </configuration>
    </execution>
  </executions>
</plugin>

```

Ⓐ The generated Q-types will be located in the `target/generated-sources/java` folder.

Ⓑ Use the `com.querydsl.apt.jpa.JPAAAnnotationProcessor` class to generate Q-types.

We'll now move on to the standard configuration file for persistence units, in `src/main/resources/META-INF/persistence.xml`. This file is shown in the following listing.

Listing 19.3 The `persistence.xml` configuration file

Path: `Ch19/querydsl/src/main/resources/META-INF/persistence.xml`

```
<persistence-unit name="ch19.querydsl">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <properties>
        <property name="javax.persistence.jdbc.driver"
            value="com.mysql.cj.jdbc.Driver"/>
        <property name="javax.persistence.jdbc.url" value="jdbc:mysql:
            ➤ localhost:3306/CH19_QUERYDSL?serverTimezone=UTC"/>
        <property name="javax.persistence.jdbc.user" value="root"/>
        <property name="javax.persistence.jdbc.password" value=""/>

        <property name="hibernate.dialect"
            value="org.hibernate.dialect.MySQL8Dialect"/>
        <property name="hibernate.show_sql" value="true"/>
        <property name="hibernate.format_sql" value="true"/>
        <property name="hibernate.hbm2ddl.auto" value="create"/>
    </properties>
</persistence-unit>
```

Ⓐ The `persistence.xml` file configures the `ch19.querydsl` persistence unit.

Ⓑ As JPA is only a specification, we need to indicate the vendor-specific `PersistenceProvider` implementation of the API. The persistence we define will be backed by a Hibernate provider.

Ⓒ The JDBC properties—the driver.

Ⓓ The URL of the database.

Ⓔ The username.

Ⓕ No password for access. The machine we are running the programs on has MySQL 8 installed, and the access credentials are the ones from `persistence.xml`. You should modify the credentials to correspond to the ones on your machine.

Ⓖ The Hibernate dialect is MySQL8, as the database we'll interact with is MySQL Release 8.0.

Ⓗ While executing, show the SQL code.

Ⓚ Hibernate will format the SQL nicely and generate comments in the SQL string so we know why Hibernate executed the SQL statement.

① Every time the program is executed, the database will be created from scratch. This is ideal for automated testing when we want to work with a clean database for every test run.

19.2.2 Creating the entities

We'll now create the classes that represent the entities of the application: `User`, `Bid`, and `Address`. The relationships between them will be of type one-to-many, many-to-one, or embedded.

Listing 19.4 The `User` class

```
Path: Ch19/querydsl/src/main/java/com/manning/javapersistence/querydsl
➡ /model/User.java
```

```
@Entity
@NoArgsConstructor
public class User {

    @Id
    @GeneratedValue(generator = Constants.ID_GENERATOR)
    @Getter
    private Long id;

    @Embedded
    @Getter
    @Setter
    private Address address;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private Set<Bid> bids = new HashSet<>();

    // . . .
}
```

Ⓐ The ID field is an identifier generated by the `Constants.ID_GENERATOR` generator. For a review of generators, revisit chapter 5.

Ⓑ The address does not have its own identity; it is embeddable.

Ⓒ There is a one-to-many relationship between `User` and `Bid`, which is mapped by the `user` field on the `Bid` side. `CascadeType.ALL` indicates that all operations will be propagated from the parent `User` to the child `Bid`.

The `Address` class does not have its own persistence identity, and it will be embeddable.

Listing 19.5 The `Address` class

```
Path: Ch19/querydsl/src/main/java/com/manning/javapersistence/querydsl
➡ /model/User.java

@Embeddable
@NoArgsConstructor
public class Address {

    //fields with Lombok annotations, constructor
}
```

The `Bid` class will contain an `id` field having a similar generation strategy as for `User`. The relationship between `Bid` and `User` will be many-to-one, not optional, and the fetch type will be lazy.

Listing 19.6 The `Bid` class

```
Path: Ch19/querydsl/src/main/java/com/manning/javapersistence/querydsl
➡ /model/Bid.java

@Entity
@NoArgsConstructor
public class Bid {

    @Id
    @GeneratedValue(generator = Constants.ID_GENERATOR)
    private Long id;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    @Getter
    @Setter
    private User user;

    // . . .

}
```

The `UserRepository` interface extends `JpaRepository<User, Long>`. It manages the `User` entity and has IDs of type `Long`. We'll use this Spring Data JPA interface only to conveniently populate the database to test Querydsl on.

Listing 19.7 The `UserRepository` interface

```
Path: Ch19/querydsl/src/main/java/com/manning/javapersistence/querydsl
➡ /repositories/UserRepository.java

public interface UserRepository extends JpaRepository<User, Long> {
```

```
}
```

19.2.3 Creating the test data to query

To populate and work with the database, we'll need a `SpringDataConfiguration` class and a `GenerateUsers` class. We have used this approach repeatedly, so we'll only quickly review the capabilities of these classes here.

Listing 19.8 The `SpringDataConfiguration` class

```
Path: Ch19/querydsl/src/test/java/com/manning/javapersistence/querydsl
➡ /configuration/SpringDataConfiguration.java

@EnableJpaRepositories("com.manning.javapersistence.querydsl.repositories")
public class SpringDataConfiguration {
    @Bean
    public DataSource dataSource() {
        // . . .
        return dataSource;
    }

    @Bean
    public JpaTransactionManager
        transactionManager(EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }

    @Bean
    public JpaVendorAdapter jpaVendorAdapter() {
        HibernateJpaVendorAdapter jpaVendorAdapter = new
            HibernateJpaVendorAdapter();
        // . . .
        return jpaVendorAdapter;
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory(
        LocalContainerEntityManagerFactoryBean
            localContainerEntityManagerFactoryBean =
                new LocalContainerEntityManagerFactoryBean();
        // . . .
        return localContainerEntityManagerFactoryBean;
    }
}
```

Ⓐ The `@EnableJpaRepositories` annotation will scan the package of the annotated configuration class for Spring Data repositories.

Ⓑ Create a data source bean to keep the JDBC properties: the driver, the URL of the database, the username, and the password.

Ⓒ Create a transaction manager bean based on an entity manager factory. Every interaction with the database should occur within transaction boundaries, and Spring Data needs a transaction manager bean.

Ⓓ Create and configure a JPA vendor adapter bean, which is needed by JPA to interact with Hibernate.

Ⓔ Create and configure a `LocalContainerEntityManagerFactoryBean`—this is a factory bean that produces an `EntityManagerFactory`.

The `GenerateUsers` class contains the `generateUsers` method, which creates a list of users and their related bids.

Listing 19.9 The `GenerateUsers` class

```
Path: Ch19/querydsl/src/test/java/com/manning/javapersistence/querydsl
➡ /GenerateUsers.java

public class GenerateUsers {

    public static Address address = new Address("Flowers Street",
                                                "1234567", "Boston", "MA");

    public static List<User> generateUsers() {
        List<User> users = new ArrayList<>();

        User john = new User("john", "John", "Smith");
        john.setRegistrationDate(LocalDate.of(2020, Month.APRIL, 13));
        john.setEmail("john@somedomain.com");
        john.setLevel(1);
        john.setActive(true);
        john.setAddress(address);

        Bid bid1 = new Bid(new BigDecimal(100));
        bid1.setUser(john);
        john.addBid(bid1);

        Bid bid2 = new Bid(new BigDecimal(110));
        bid2.setUser(john);
        john.addBid(bid2);

        // . . .
    }
}
```


19.3 Querying a database with Querydsl

As we mentioned previously, the Maven APT plugin will generate the Q-types during the build process. According to the provided configuration (see listing 19.2), these sources will be generated in the `target/generated-sources/java` folder (see figure 19.1). We'll use these generated classes to query the database.

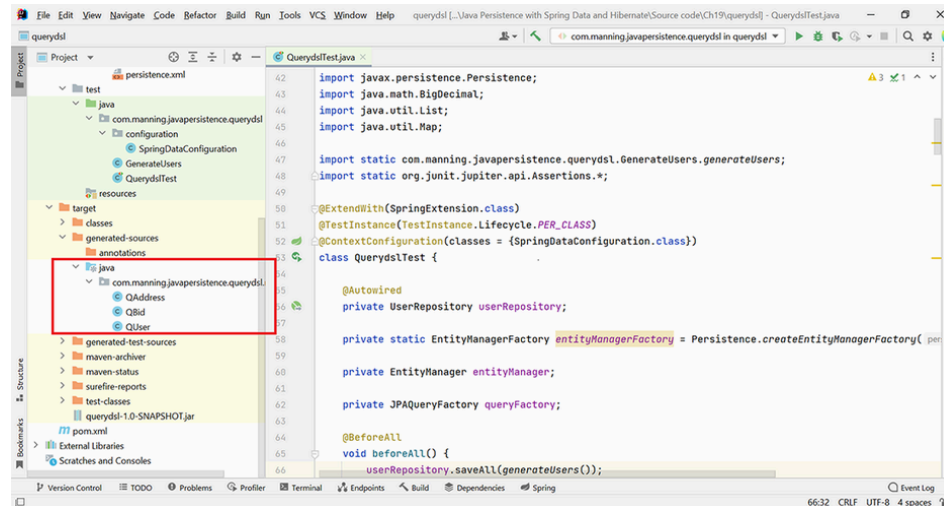


Figure 19.1 The generated Q-types in the `target` folder

First, though, we have to populate the database, and to do this, we'll use the `UserRepository` interface. We'll also use an `EntityManagerFactory` and the created `EntityManager` to start working with a `JPAQueryFactory` and a `JPAQuery`. We'll need a `JPAQueryFactory` instance to work with queries, and it will be created by the constructor that takes an `EntityManager` argument. Then, `JPAQueryFactory` will create `JPAQuery` instances, to effectively query the database.

We'll extend the test using `SpringExtension`. This extension is used to integrate the Spring test context with the JUnit 5 Jupiter test.

Before executing the tests, we'll populate the database with previously generated users and their corresponding bids. Before each test, we'll create an `EntityManager` and start a transaction. Thus, every interaction with the database will occur within transaction boundaries. For the moment, we won't execute queries from inside this class, but as tests and queries will be immediately added (starting with listing 19.11), we'll name our class `QuerydslTest`.

Listing 19.10 The `QuerydslTest` class

```
Path: Ch19/querydsl/src/test/java/com/manning/javapersistence/querydsl
➡ /QuerydslTest.java
```

```

@ExtendWith(SpringExtension.class)
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
@ContextConfiguration(classes = {SpringDataConfiguration.class})
class QuerydslTest {

    @Autowired
    private UserRepository userRepository;

    private static EntityManagerFactory entityManagerFactory =
        Persistence.createEntityManagerFactory("ch19.querydsl")

    private EntityManager entityManager;

    private JPAQueryFactory queryFactory;

    @BeforeAll
    void beforeAll() {
        userRepository.saveAll(generateUsers());
    }

    @BeforeEach
    void beforeEach() {
        entityManager = entityManagerFactory.createEntityManager();
        entityManager.getTransaction().begin();
        queryFactory = new JPAQueryFactory(entityManager);
    }

    @AfterEach
    void afterEach() {
        entityManager.getTransaction().commit();
        entityManager.close();
    }

    @AfterAll
    void afterAll() {
        userRepository.deleteAll();
    }
}

```

Ⓐ Extend the test using `SpringExtension`.

Ⓑ JUnit will create only one instance of the test class for executing all tests, instead of one instance per test. This way we'll be able to autowire the `UserRepository` field as an instance variable.

Ⓒ The Spring test context is configured using the beans defined in the previously presented `SpringDataConfiguration` class.

Ⓓ A `UserRepository` bean is injected by Spring through autowiring. It will be used to easily populate and clean up the database.

- Ⓔ Initialize an `EntityManagerFactory` to talk to the database. This one will create the `EntityManager` that is needed by the `JPAQueryFactory`.
- Ⓕ Declare the `EntityManager` and the `JPAQueryFactory` needed for the application.
- Ⓖ Populate the database with previously generated users and bids, to be used by the tests.
- Ⓗ Create a `JPAQueryFactory` by passing an `EntityManager` as an argument to its constructor.
- Ⓘ At the end of each test, commit the transaction and close the `EntityManager`.
- Ⓢ At the end of the execution of all tests, clean up the database.

19.3.1 Filtering data

As previously discussed, the entities that are managed by the application will be replicated in the so-called Q-types. This means that each `Entity` entity will have a corresponding `QEntity` that will be generated at build time, which Querydsl will use to query the database. The Q-type classes generated by the Maven APT plugin each contain a static instance of their kind:

```
public static final QUser user = new QUser("user");
public static final QBid bid = new QBid("bid");
public static final QAddress address = new QAddress("address");
```

These instances will be used to query the database. We'll first get a `JPAQuery` instance by calling `queryFactory.selectFrom(user)`. Then we'll use this `JPAQuery` instance to build the clauses of the query. We'll use the `where` method to filter by a given `Predicate` and the `fetchOne` method to fetch a single element from the database. `fetchOne` returns `null` if no element to fulfill a condition is found, and it throws `NonUniqueResultException` if multiple elements that fulfill a condition are found.

For example, to get the `User` with a given `username`, we'll write the code in the following listing.

Listing 19.11 Finding a `User` by `username`

```
Path: Ch19/querydsl/src/test/java/com/manning/javapersistence/querydsl
➡ /QuerydslTest.java
```

```

@Test
void testFindByUsername() {

    User fetchedUser = queryFactory.selectFrom(QUser.user)
                                    .where(QUser.user.username.eq("john"))
                                    .fetchOne();

    assertEquals("John", fetchedUser.getFirstName());
    assertEquals("Smith", fetchedUser.getLastName());
    assertEquals(2, fetchedUser.getBids().size());

}

```

Ⓐ Start building the query using the `selectFrom` method belonging to the `JPAQueryFactory` class. This method will get the created Q-type instance `QUser.user` as an argument and will return a `JPAQuery`.

Ⓑ The `where` method will filter by the given `Predicate` related to the `username`.

Ⓒ The `fetchOne` method will try to fetch a single element from the database.

Ⓓ Verify that the fetched data is the expected data.

The following SQL queries are generated by Hibernate:

```

select
    *
from
    User user0_
where
    user0_.username=?

select
    *
from
    Bid bids0_
where
    bids0_.user_id=?

```

We can filter by multiple `Predicate`s using methods such as `and` or `or`, which each receive a `Predicate`. For example, to filter by the `level` and `active` fields, we could write the following piece of code:

```
List<User> users = (List<User>)queryFactory.from(QUser.user)
    .where(QUser.user.level.eq(3)
    .and(QUser.user.active.eq(true))).fetch()
```

The following SQL query is generated by Hibernate:

```
select
    *
from
    User user0_
where
    user0_.level=?
    and user0_.active=?
```

19.3.2 Ordering data

To order data, we'll use the `orderBy` method, which can receive multiple arguments representing the criteria to order by. For example, to get the `User` instances ordered by `username`, we'll write the following code.

Listing 19.12 Ordering `User` instances by `username`

```
Path: Ch19/querydsl/src/test/java/com/manning/javapersistence/querydsl
➡ /QuerydslTest.java
```

```
@Test
void testOrderByUsername() {

    List<User> users = queryFactory.selectFrom(QUser.user)
        .orderBy(QUser.user.username.asc())
        .fetch();

    assertAll(
        () -> assertEquals(users.size(), 10),
        () -> assertEquals("beth", users.get(0).getUsername()),
        () -> assertEquals("burk", users.get(1).getUsername()),
        () -> assertEquals("mike", users.get(8).getUsername()),
        () -> assertEquals("stephanie", users.get(9).getUsername())
    );
}
```

Ⓐ Start building the query using the `selectFrom` method belonging to the `JPAQueryFactory` class. This method will get the created Q-type instance `QUser.user` as an argument and will return a `JPAQuery`.

Ⓑ Order the results by `username`, ascending. The `orderBy` method is overloaded and can receive multiple criteria to order by.

Ⓒ The `fetch` method will fetch the list of `User` instances.

- ① Verify that the fetched data is the expected data.

The following SQL query is generated by Hibernate:

```
select
    *
from
    User user0_
order by
    user0_.username asc
```

19.3.3 Grouping data and working with aggregates

To group data, we'll use the `groupBy` method, which receives the expression to group by. Such a query will return a `List<Tuple>`. A `com.querydsl.core.Tuple` object is a key/value pair that contains the key to group by and its corresponding value. For example, the following code counts `Bid` instances grouped by `amount`.

Listing 19.13 Grouping bids by amount

```
Path: Ch19/querydsl/src/test/java/com/manning/javapersistence/querydsl
➡ /QuerydslTest.java

@Test
void testGroupByBidAmount() {

    NumberPath<Long> count = Expressions.numberPath(Long.class, "bids");

    List<Tuple> userBidsGroupByAmount =
        queryFactory.select(QBid.bid.amount,
                           QBid.bid.id.count().as(count))
                    .from(QBid.bid)
                    .groupBy(QBid.bid.amount)
                    .orderBy(count.desc())
                    .fetch();

    assertAll(
        () -> assertEquals(new BigDecimal("120.00"),
                           userBidsGroupByAmount.get(0).get(QBid.bid.amount)),
        () -> assertEquals(2, userBidsGroupByAmount.get(0).get(count))
    );
}
```

- Ⓐ Keep the `count()` expression, as we'll need it multiple times later when building the query.

- Ⓑ Return a `List<Tuple>`, the key/value pairs with the key to group by and its corresponding value.

- Ⓒ Select the `amount` and the count of the same `amount` s from `Bid`.
- Ⓓ Group by the `amount` value.
- Ⓔ Order by the count of records having the same amount.
- Ⓕ Fetch the `List<Tuple>` objects.
- Ⓖ Verify that the fetched data is the expected data.

The following SQL query is generated by Hibernate:

```
select
    bid0_.amount as col_0_0_,
    count(bid0_.id) as col_1_0_
from
    Bid bid0_
group by
    bid0_.amount
order by
    col_1_0_ desc
```

To work with aggregation and get the maximum, minimum, and average values of the `Bids`, we can use the `max`, `min`, and `avg` methods, as in the following code:

```
queryFactory.from(QBid.bid).select(QBid.bid.amount.max()).fetchOne();
queryFactory.from(QBid.bid).select(QBid.bid.amount.min()).fetchOne();
queryFactory.from(QBid.bid).select(QBid.bid.amount.avg()).fetchOne();
```

The following SQL queries are generated by Hibernate:

```
select
    max(bid0_.amount) as col_0_0_
from
    Bid bid0_

select
    min(bid0_.amount) as col_0_0_
from
    Bid bid0_

select
    avg(bid0_.amount) as col_0_0_
from
    Bid bid0_
```

19.3.4 Working with subqueries and joins

To work with subqueries, we'll create a subquery with the `JPAExpressions` static factory methods (such as `select`), and we'll define the query parameters using methods such as `from` and `where`. We'll pass the subquery to the `where` method of the main query. For example, the following listing selects the `User`s having `Bid`s with a given `amount`.

Listing 19.14 Working with subqueries

```
Path: Ch19/querydsl/src/test/java/com/manning/javapersistence/querydsl
➡ /QuerydslTest.java

@Test
void testSubquery() {

    List<User> users = queryFactory.selectFrom(QUser.user)
        .where(QUser.user.id.in(
            JPAExpressions.select(QBid.bid.user.id)
                .from(QBid.bid)
                .where(QBid.bid.amount.eq(
                    new BigDecimal("120.00")))))
        .fetch();

    List<User> otherUsers = queryFactory.selectFrom(QUser.user)
        .where(QUser.user.id.in(
            JPAExpressions.select(QBid.bid.user.id)
                .from(QBid.bid)
                .where(QBid.bid.amount.eq(
                    new BigDecimal("105.00")))))
        .fetch();

    assertAll(
        () -> assertEquals(2, users.size()),
        () -> assertEquals(1, otherUsers.size()),
        () -> assertEquals("burk", otherUsers.get(0).getUsername())
    );
}
```

Ⓐ Start building the query using the `selectFrom` method belonging to the `JPAQueryFactory` class.

Ⓑ Pass the subquery as a parameter of the `where` method.

Ⓒ Create the subquery to get the `Bid`s having `amount` 120.00.

Ⓓ Fetch the result.

Ⓔ Create a similar query and subquery for the `Bid`s having `amount` 105.00.

Ⓕ Verify that the fetched data is as expected.

The following SQL query is generated by Hibernate:

```
select
    *
from
    User user0_
where
    user0_.id in (
        select
            bid1_.user_id
        from
            Bid bid1_
        where
            bid1_.amount=?
    )
```

To work with joins, we'll use the methods `innerJoin`, `leftJoin`, and `outerJoin` to define the join, and we'll use the `on` method to declare the condition (a `Predicate`) to join on.

Listing 19.15 Working with joins

Path: Ch19/querydsl/src/test/java/com/manning/javapersistence/querydsl
➡ /QuerydslTest.java

```
@Test
void testJoin() {

    List<User> users = queryFactory.selectFrom(QUser.user)
        .innerJoin(QUser.user.bids, QBid.bid)
        .on(QBid.bid.amount.eq(new BigDecimal("120.00")))
        .fetch();

    List<User> otherUsers = queryFactory.selectFrom(QUser.user)
        .innerJoin(QUser.user.bids, QBid.bid)
        .on(QBid.bid.amount.eq(new BigDecimal("105.00")))
        .fetch();

    assertEquals(2, users.size());
    assertEquals(1, otherUsers.size());
    assertEquals("burk", otherUsers.get(0).getUsername());
}
```

Ⓖ Start building the query using the `selectFrom` method belonging to the `JPAQueryFactory` class.

- Ⓑ Make the inner join to `Bids`.
- Ⓒ Define the join on condition as a `Predicate` to have the `Bid` amount 120.00.
- Ⓓ Fetch the result.
- Ⓔ Create a similar join for the `Bids` having amount 105.00.
- Ⓕ Verify that the fetched data is as expected.

The following SQL query is generated by Hibernate:

```
select
    *
from
    User user0_
inner join
    Bid bids1_
    on user0_.id=bids1_.user_id
    and (
        bids1_.amount=?
    )
```

19.3.5 Updating entities

To update the entities, we'll use the `update` method of the `JPAQueryFactory` class, the `where` method to define the `Predicate` that will filter the entities to be updated (optional), the `set` method to define the changes to be made, and the `execute` method to effectively execute the update.

Listing 19.16 Updating the information

Path: `Ch19/querydsl/src/test/java/com/manning/javapersistence/querydsl`
 ➡ `/QuerydslTest.java`

```
@Test
void testUpdate() {

    queryFactory.update(QUser.user)
        .where(QUser.user.username.eq("john"))
        .set(QUser.user.email, "john@someotherdomain.com")
        .execute();

    entityManager.getTransaction().commit();

    entityManager.getTransaction().begin();

    assertEquals("john@someotherdomain.com",
        queryFactory.select(QUser.user.email)
```

```

        .from(QUser.user)
        .where(QUser.user.username.eq("john"))
        .fetchOne());
    }
}

```

- Ⓐ Start building the query using the `update` method belonging to the `JPAQueryFactory` class.
- Ⓑ Define the `where` condition to update on (optional).
- Ⓒ Define the changes to be made to the entities with the `set` method.
- Ⓓ Effectively execute the update.
- Ⓔ Commit the transaction that was started in the `@BeforeEach` annotated method.
- Ⓕ Start a new transaction that will be committed in the `@AfterEach` annotated method.
- Ⓖ Check the result of the update by fetching the modified entity.

The following SQL query is generated by Hibernate:

```

update
  User
set
  email=?
where
  username=?

```

19.3.6 Deleting entities

To delete entities, we'll use the `update` method of the `JPAQueryFactory` class, the `where` method to define the `Predicate` that will filter the entities to be deleted (optional), and the `execute` method to effectively execute the delete.

There's a big problem with Querydsl here. In the Querydsl reference documentation

(<http://querydsl.com/static/querydsl/latest/reference/html/>), section 2.1.11 on `DELETE` queries using JPA notes, "DML clauses in JPA don't take JPA level cascade rules into account and don't provide fine-grained second level cache interaction." Therefore, the cascade attribute of the `@OneToMany` annotation in the `User` class is ignored; it is necessary to select a user and manually delete their bids before deleting it through a Querydsl delete query.

As an extra proof that the issue comes from Querydsl, if a user was to be deleted through a `userRepository.delete(burk);` instruction, the `@OneToMany` cascade attribute would be properly taken into account, and no manual handling of the user's bids would be needed.

Listing 19.17 Deleting the information

Path: Ch19/querydsl/src/test/java/com/manning/javapersistence/querydsl
➡ /QuerydslTest.java

```
@Test
void testDelete() {

    User burk = (User) queryFactory.from(QUser.user)
        .where(QUser.user.username.eq("burk"))
        .fetchOne();
    if (burk != null) {
        queryFactory.delete(QBid.bid)
            .where(QBid.bid.user.eq(burk))
            .execute();
    }

    queryFactory.delete(QUser.user)
        .where(QUser.user.username.eq("burk"))
        .execute();

    entityManager.getTransaction().commit();
    entityManager.getTransaction().begin();
    assertNull(queryFactory.selectFrom(QUser.user)
        .where(QUser.user.username.eq("burk"))
        .fetchOne());
}
```

- Ⓐ Find user `burk`.
- Ⓑ Delete the bids belonging to the previously found user.
- Ⓒ Start building the query using the `delete` method belonging to the `JPAQueryFactory` class.
- Ⓓ Define the `where` condition to delete on (optional).
- Ⓔ Effectively execute the delete.
- Ⓕ Commit the transaction that was started in the `@BeforeEach` annotated method.
- Ⓖ Start a new transaction that will be committed in the `@AfterEach` annotated method. This is why in this listing a `commit` transaction appears before a `begin` transaction.

④ Check the result of the delete by trying to fetch the entity. If the entity no longer exists, the `fetchOne` method will return `null`.

The following SQL query is generated by Hibernate:

```
delete
from
    User
where
    username=?
```

This query is not enough if the `User` has child `Bids` that need to be deleted first. This reveals another problem specific to MySQL: when Hibernate creates the schema, no `ON DELETE CASCADE` clause is added when defining the foreign key constraint on the `user_id` column in the `bid` table. Otherwise, this single `DELETE` query would be sufficient regardless of Querydsl ignoring `@OneToMany` cascade attributes.

No insert capability is provided by the Querydsl API. For inserting entities, you may use an `EntityManager` (JPA), `Session` (Hibernate), or repository (Spring Data JPA).

Summary

- Query alternatives such as SQL, JPQL, Criteria API, Spring Data have drawbacks: lack of portability, lack of type safety and static query verification, and verbosity. Querydsl addresses the important ideas of type safety and portability, and it reduces the verbosity.
- You can create a persistence application to use Querydsl, define its configuration and entities, and persist and query data.
- To work with Querydsl, its dependencies need to be added to the application. The Maven APT (Annotation Processing Tool) is required for the creation, at build time, of the Q-types that replicate the entities.
- You can work with the core Querydsl classes, `JPAQueryFactory` and `JPAQuery`, to build the queries to retrieve, update, and delete data.
- You can create queries to filter, order, and group data and queries to execute joins, updates, and deletes.